

Kubernetes Upgrades & Versioning

Why Kubernetes Upgrade is Important?

- 👉 Kubernetes Upgrade means updating our cluster to a newer version to get:
 - 🛡 Security patches
 - 🐛 Bug fixes
 - ✨ New features
 - ⚡ Performance improvements

Why Upgrades are Important?

- 🔥 Benefits:
 - Stay secure from vulnerabilities
 - Avoid unsupported versions
 - Access new Kubernetes features
 - Maintain cluster stability

👉 Best Practice: Stay 1–2 versions behind latest to remain stable & supported.

Kubernetes Versioning

Kubernetes follows Semantic Versioning :

[major].[minor].[patch]

Example: 1.35.1

Version Meaning:

 Version Type	 Description	 What it Means
 Major	 Rare breaking changes	 Big changes , may break compatibility
 Minor	✨ New features (~every 4 months)	 Regular updates with new capabilities

 Version Type	 Description	 What it Means
 Patch	 Bug fixes & security updates	 Safe updates , no breaking changes



Supported Versions

-  Kubernetes supports  last 3 minor versions only
- Example:
 - 1.35 (released on January 28, 2026)
 - 1.34
 - 1.33
 - 1.32  (deprecated soon)



Amazon EKS Version Support

-  Standard support → 14 months
-  Extended support → +12 months (paid extra cost)



Upgrade Order (Important)

 Wrong: NOT allowed

1.27 → 1.29

 Correct: You MUST upgrade step-by-step

1.27 → 1.28 → 1.29

-  Always upgrade one minor version at a time



EKS Cluster Upgrade (AWS Console UI) – Step-by-Step Guide

Overview

- Upgrading an EKS cluster via the AWS Management Console (UI) involves 3 main parts:
 - **1** Control Plane
 - **2** Worker Nodes (Node Groups)
 - **3** Add-ons (CoreDNS , kube-proxy , VPC CNI)
 - 👉 Goal: ✅ Upgrade safely with zero downtime

Prerequisites for EKS Upgrade

- Before upgrading:
 - ✅ Cluster is in supported version
 - 💾 Take Backup important data
 - ↺ Use Deployments/StatefulSets
 - 🛡 Apply PodDisruptionBudgets
 - ⚠ No failing pods (`kubectl get pods`)
-

◆ Step 1: Login to AWS Console

1. Go to: <https://console.aws.amazon.com/eks>
2. Select your Region (top-right)
3. Click your Cluster Name

◆ Step 2: Check Current Version

- Go to Overview Tab
- Check Kubernetes Version (e.g., 1.34)

◆ Step 3: Upgrade Control Plane

1. Click "Update now" beside version
2. Select next minor version (e.g., 1.34 → 1.35)
3. Click Update

What Happens?

- Control plane components upgraded: API Server , Scheduler and Controller Manager
- ⌚ Takes ~10-20 minutes & ✅ Workloads not immediately affected

◆ Step 4: Upgrade Node Groups

1. Go to Compute Tab
2. Select Node Group
3. Click Update version
4. Choose same version as control plane
5. Click Update

📌 AWS performs:

- 🔄 Rolling Upgrade Flow:
 - 1 New nodes are created
 - 2 Old node is cordoned
 - 3 Pods are drained (evicted)
 - 4 Pods move to new nodes
 - 5 Old node terminated
 - ✅ Zero downtime (if PDB configured)

◆ Step 5: Upgrade Add-ons

- Go to Add-ons Tab
- 👉 Update each:
 - 🌐 Amazon VPC CNI (aws-node)
 - 🌐 CoreDNS
 - 🔄 kube-proxy
 - 🧭 Steps:
 - i. Click Update
 - ii. Select latest compatible version
 - iii. Click Next → Update



Why Important?

- Ensures compatibility with new Kubernetes version
- Avoids networking/DNS issues

◆ Step 6: Verify Cluster Health

Check:  Nodes &  Pods

All nodes = ****Ready****

All pods = ****Running / Completed****



CLI Verification:

```
kubectl get nodes
```

```
kubectl get pods -A
```



Monitoring

- CloudWatch Logs
- Alerts
- Prometheus / Grafana



Step 7: Smoke Testing (CRITICAL)

- 👉 Quickly test key features :
 - ✎ Examples:
 - Open application URL 
 - Call APIs 
 - Check DB connectivity 
 - Verify login/payment flows 
 - ✔ Ensures everything works after upgrade (No errors in logs)
-

why Kubernetes upgrades:

-  Keep cluster secure & updated

- ⚡ Improve performance
- 🚀 Add new features
- 🛡️ Maintain compatibility (exist together without problems or conflict).

🚀 Best Practices

- 🛠️ Upgrade in non-prod first
- 📊 Monitor cluster health:
 - `kubectl get cs`
 - Prometheus
- 🛑 Use PodDisruptionBudgets (PDB)
- 🔄 Follow staging → production strategy

⚙️ Safe Upgrade Strategy

- 1 Upgrade control plane
- 2 Upgrade node groups
- 3 Verify workloads
- 4 Monitor logs & metrics
- 5 Roll forward (or rollback if needed)

⚠️ Risks Without Proper Upgrade

- ❌ Downtime
- ❌ API deprecations
- ❌ Unsupported features
- ❌ Security vulnerabilities

⚠️ Important Notes

- Upgrade one version at a time
- Always test in staging first
- Use PodDisruptionBudgets
- Monitor during upgrade

✨ Summary

EKS Upgrade Flow:

- 1 Control Plane
- 2 Node Groups
- 3 Add-ons
- 4 Verification

👉 Safe upgrades = Zero downtime 🚀